

MAALI Partner API — Client Integration Specification

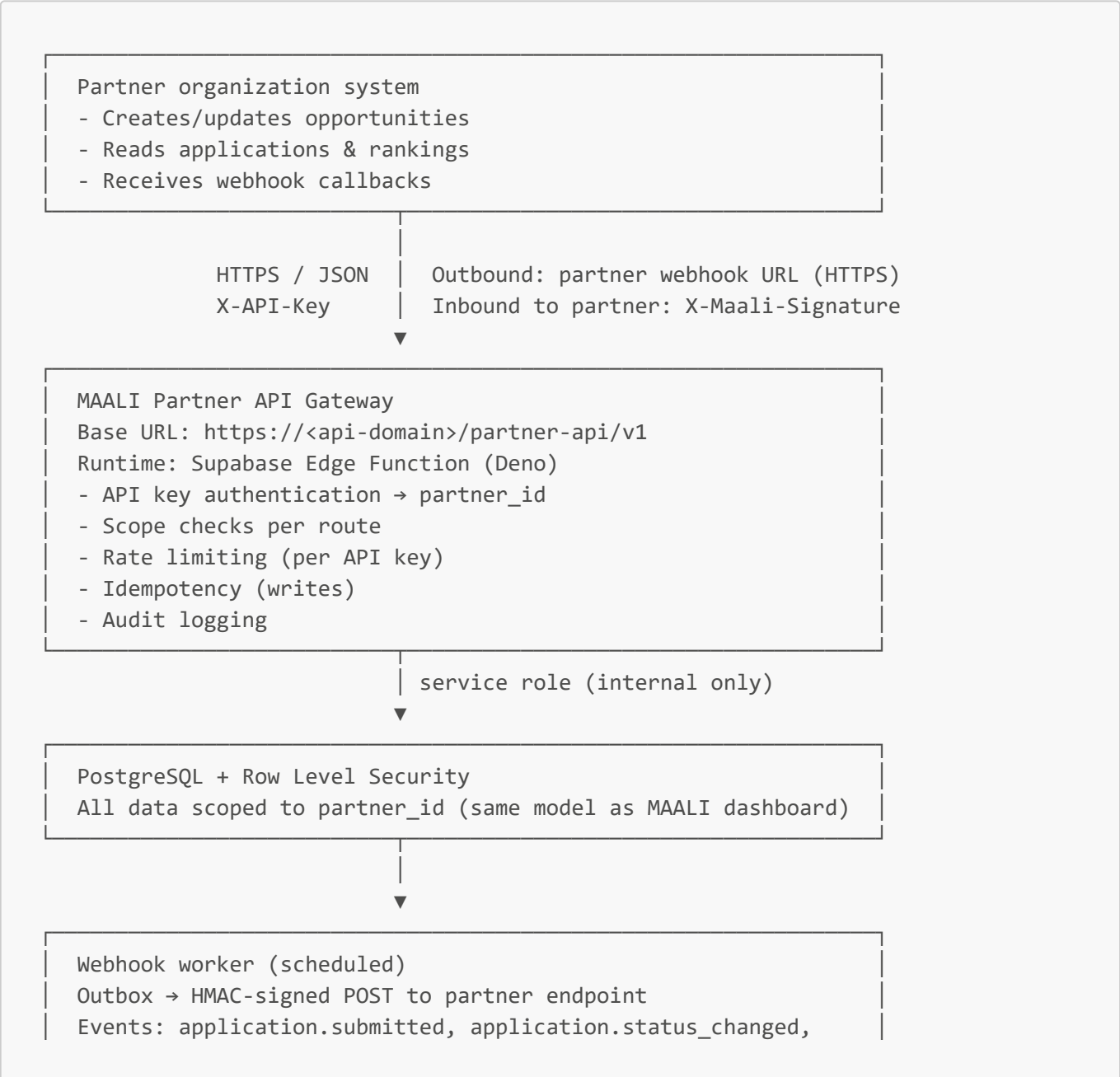
Document purpose: Technical summary for a partner organization (integrator) connecting their CRM/ATS/grants system to MAALI.

Production hosting (MAALI): Self-hosted on VPS. Partner systems connect over public HTTPS only; they do not access MAALI infrastructure directly.

Status: Partner API v1 is complete for production use with live keys. A dedicated sandbox (isolated data, write testing, sandbox webhooks) is in development. Until then, use read-only `mpk_test_` keys against your organization data for integration testing.

1. System architecture

1.1 Integration context



opportunity.closed

1.2 What the partner system must support

Requirement	Specification
Protocol	HTTPS only (TLS 1.2+)
Request format	JSON, UTF-8, <code>Content-Type: application/json</code>
Authentication	Header <code>X-API-Key: mpk_live_...</code> or <code>mpk_test_...</code>
Write idempotency	Header <code>Idempotency-Key</code> on POST/PATCH writes (recommended UUID)
Webhook endpoint	Public HTTPS URL; must verify <code>X-Maali-Signature</code>
Clock	Webhook verification requires ± 5 minute timestamp tolerance
Retries	Partner must tolerate at-least-once webhooks; dedupe on <code>event.id</code>

2. API codes (machine-readable reference)

Codes below are what the integrator must implement against. Stable within API version `/v1`.

2.1 API key prefixes

Prefix	Meaning
<code>mpk_test_</code>	Read-only integration credential (GET summary only; no PII; see limitations below)
<code>mpk_live_</code>	Production credential (read and write per assigned scopes)

Keys are stored by MAALI as SHA-256 hash only. Full secret shown once at creation.

Test key limitations

Test keys are **read-only**. They are not a separate sandbox dataset.

Rule	Detail
Writes blocked	All mutating routes return <code>403</code> + <code>errorCode: TEST_KEY_READONLY</code> .
No PII	Applicant PII and signed document URLs are never returned; <code>applications:read_pii</code> not allowed on test keys.
Allowed read scopes	<code>opportunities:read</code> , <code>applications:read</code> only.
Same dataset	Test and live keys read the same <code>partner_id</code> organization data.
Webhooks	Configure via Partner portal or a live key — not via test key API calls.
Intended use	Safe integration testing of read flows; live keys for production writes.

2.2 Authorization scopes

Scope code	Grants
<code>opportunities:read</code>	List/get opportunities; GET organization
<code>opportunities:write</code>	Create, update, close opportunities; PATCH organization
<code>applications:read</code>	List/get applications (summary, ranked)
<code>applications:read_pii</code>	Applicant PII + signed document URLs
<code>webhooks:manage</code>	GET/PUT <code>/webhooks</code> , POST <code>/webhooks/rotate-secret</code>

2.3 HTTP API error codes (`errorCode`)

Response shape: { "error": "<human message>", "errorCode": "<CODE>" }

HTTP	errorCode	Meaning
400	<code>VALIDATION_ERROR</code>	Invalid JSON, body, or query params
401	<code>UNAUTHORIZED</code>	Missing or invalid API key
401	<code>KEY_EXPIRED</code>	Key revoked or past <code>expires_at</code>
403	<code>FORBIDDEN_SCOPE</code>	Key lacks required scope
403	<code>TEST_KEY_READONLY</code>	Test API key used on a write route
404	<code>NOT_FOUND</code>	Resource not found or not owned by partner
409	<code>IDEMPOTENCY_CONFLICT</code>	Same <code>Idempotency-Key</code> reused with different body
409	<code>IDEMPOTENCY_IN_PROGRESS</code>	Same idempotency key already processing
429	<code>RATE_LIMITED</code>	Rate limit exceeded
500	<code>INTERNAL_ERROR</code>	Server error

2.4 Rate limit headers

Header	Description
<code>X-RateLimit-Remaining</code>	Requests remaining in current window
<code>X-RateLimit-Reset</code>	ISO 8601 reset time
<code>Retry-After</code>	Seconds (on 429 only)

Limits per API key: 20 requests/minute; 1,000 successful requests/month (2xx only; see §3).

2.5 Webhook event type codes

Event code	When fired
------------	------------

Event code	When fired
<code>application.submitted</code>	Non-draft application submitted to partner opportunity
<code>application.status_changed</code>	Application status changes
<code>opportunity.closed</code>	Opportunity closed

2.6 Webhook signature header

Header	Format
<code>X-Maali-Signature</code>	<code>t=<unix_seconds>,v1=<hex_hmac_sha256></code>
<code>X-Maali-Event-Id</code>	Event id (same as payload <code>id</code>)

Verification: `v1 = HMAC_SHA256(signing_secret, "<t>.<raw_body>")`

2.7 Opportunity status values (API)

Value	Description
<code>open</code>	Accepting applications
<code>closed</code>	Closed
<code>archived</code>	Archived

2.8 REST endpoints summary

Method	Path	Required scope
GET	<code>/organization</code>	<code>opportunities:read</code>
PATCH	<code>/organization</code>	<code>opportunities:write</code>
GET	<code>/opportunities</code>	<code>opportunities:read</code>
GET	<code>/opportunities/{id}</code>	<code>opportunities:read</code>
POST	<code>/opportunities</code>	<code>opportunities:write</code>
PATCH	<code>/opportunities/{id}</code>	<code>opportunities:write</code>
POST	<code>/opportunities/{id}/close</code>	<code>opportunities:write</code>
GET	<code>/opportunities/{id}/applications</code>	<code>applications:read</code>
GET	<code>/applications/{appId}</code>	<code>applications:read</code> (+ PII if scoped)
GET	<code>/webhooks</code>	<code>webhooks:manage</code>
PUT	<code>/webhooks</code>	<code>webhooks:manage</code>
POST	<code>/webhooks/rotate-secret</code>	<code>webhooks:manage</code>

2.9 Success response envelope

```
{ "data": { ... } }
```

List endpoints also return:

```
{ "data": [ ... ], "next_cursor": "<opaque>|null" }
```

Cursor pagination (list endpoints only, e.g. `GET /opportunities`):

Query param	Description
<code>limit</code>	Page size (default 50, max 100)
<code>cursor</code>	Opaque token from previous response <code>next_cursor</code>

First page — omit `cursor`:

```
GET /partner-api/v1/opportunities?limit=50
```

Next page — pass `next_cursor` verbatim as `cursor`:

```
GET /partner-api/v1/opportunities?  
limit=50&cursor=eyJpZCI6MSwiY3JlYXRlZF9hdCI6Li4ufQ==
```

Repeat until `next_cursor` is `null`. Treat `cursor` as opaque; do not decode or construct it client-side.

3. Monthly transaction limit

3.1 Definition of a transaction

One transaction = one **successful inbound** Partner API request (HTTP **2xx**) from the partner system to MAALI.

Not counted as transactions:

- Outbound webhooks (MAALI → partner)
- Failed requests (4xx / 5xx)
- Retries that fail
- Partner portal UI usage

3.2 Hard cap

Limit	Monthly	Annual
-------	---------	--------

Limit	Monthly	Annual
Successful requests (2xx) per API key	1,000	~12,000 (12 × monthly cap)
Burst rate per API key	20 / minute	—

Cap enforced on a **rolling monthly** window per key, not as a single annual bucket. Once the monthly cap is reached, further requests return 429 + `errorCode: RATE_LIMITED` until the next monthly window.

Outbound webhooks are not counted against this cap.